

Materia: Programación III

Carrera: Tecnicatura Universitaria en Programación (UTN)

Tema: Arquitectura Cliente-Servidor, Transición de C# a JavaScript y Programación Asíncrona

CLASE 1: Arquitectura Cliente-Servidor y fundamentos de JavaScript

1. Arquitectura Cliente-Servidor (Conceptos Fundamentales)

En las materias anteriores (Programación I y II), las aplicaciones ejecutaban toda su lógica, persistencia e interfaz gráfica en una sola máquina (aplicaciones de consola o de escritorio monolíticas). En **Programación III** dividimos el sistema en dos roles principales:

- **El Servidor (Backend):** Responsable de la lógica de negocio, acceso a la base de datos (mediante un ORM) y la seguridad de los datos. Expone servicios (APIs) a los cuales los clientes se conectan.
- **El Cliente (Frontend / Mobile):** Responsable de la interfaz gráfica de usuario (UI), la interacción con el usuario y la presentación visual de la información. En nuestro curso, la aplicación móvil desarrollada con React Native.
- **Protocolo de Comunicación (HTTP / HTTPS):** El canal por el cual se transfieren los datos entre el cliente y el servidor en formato JSON.

CLIENTE (Mobile) (React Native / JavaScript)	Petición HTTP ----->	SERVIDOR (Backend) (.NET / C# Web API)
Interfaz de Usuario (UI)	<----- Respuesta JSON	Lógica + Base de Datos

2. Introducción a JavaScript (JS)

JavaScript es el lenguaje estándar del desarrollo web y móvil multiplataforma (React Native). A diferencia de C#, JavaScript es un lenguaje interpretado de tipado dinámico. Esto significa que las variables no requieren especificar su tipo de dato al declararse.

A. Declaración de Variables

Concepto	C#	JavaScript
Texto	string nombre = "Juan";	let nombre = "Juan";
Números	int edad = 20;	let edad = 20;

Concepto	C#	JavaScript
	float precio = 9.99f;	let precio = 9.99;
Booleano	bool activo = true;	let activo = true;
Inmutabilidad / Constantes	const int MAX = 100;	const MAX = 100;
Variables mutables	int x = 10;	let x = 10; <i>(Evitar el uso de var)</i>

B. Objetos y Estructuras de Datos

En C# definían clases o structs para modelar entidades. En JavaScript utilizaremos **Objetos Literales** para representar estructuradamente los datos recibidos en formato JSON desde el servidor:

C#:

```
public class Usuario {
    public int Id { get; set; }
    public string Nombre { get; set; }
    public string Email { get; set; }
}

Usuario usuarioActual = new Usuario { Id = 1, Nombre = "Osvaldo" };
```

JavaScript:

```
// Creación directa de objeto literal (JS no requiere definir clase previa)
const usuarioActual = {
    id: 1,
    nombre: "Osvaldo",
    email: "osvaldo@ejemplo.com"
};
```

C. Funciones Flecha (Arrow Functions)

En C# usaban métodos tradicionales o expresiones lambda (`=>`). En JavaScript, las **funciones flecha** son el estándar moderno para definir funciones:

C#:

```
public int Sumar(int a, int b) {  
    return a + b;  
}
```

JavaScript:

```
const sumar = (a, b) => {  
    return a + b;  
};  
  
// Sintaxis simplificada de una sola línea:  
const sumarCorta = (a, b) => a + b;
```

D. Transformación de Colecciones: Arreglos (Arrays) vs. LINQ

En C# utilizaban **LINQ** (`.Select()`, `.Where()`, `.FirstOrDefault()`). En JavaScript utilizaremos métodos nativos de los arreglos:

```
const numeros = [1, 2, 3, 4, 5];  
  
// 1. Filtrar elementos mayores a 2 (Equivalente a .Where en LINQ)  
const mayoresA2 = numeros.filter(n => n > 2); // [3, 4, 5]  
  
// 2. Transformar elementos (Equivalente a .Select en LINQ)  
const dobles = numeros.map(n => n * 2); // [2, 4, 6, 8, 10]  
  
// 3. Buscar el primer elemento que cumpla la condición (Equivalente  
a .FirstOrDefault en LINQ)  
const primerPar = numeros.find(n => n % 2 === 0); // 2
```